

У меня есть игра https://github.com/AlexanderRozov/Cross-Bound/tree/v_0_0_1 Ты должен написать все классы которые необходимы чтобы обеспечить подключение <https://yandex.ru/dev/games/doc/ru/sdk/unity/install> Вот документация <https://max-games.ru/plugin-yg/doc/>

Да. Я посмотрел именно ветку `v_0_0_1` твоего `Cross-Bound`, а также актуальную документацию Яндекс Игр и Plugin Your Games 2.0.

У тебя уже есть хорошая точка входа: `ProjectStart`, `PlayerProfileManager`, `SceneController`, `CrosswordGameController`. Я бы не встраивал `yg2` непосредственно в игровые классы. Лучше сделать отдельный слой `Platform/Yandex`, чтобы игра могла работать:

- в Unity Editor;
- в обычном WebGL;
- на Яндекс Играх;
- с авторизацией;
- с облачными сохранениями;
- с рекламой;
- с лидербордом;
- с `GameReady`;
- с `GameplayStart/GameplayStop`.

`PluginYG2` как раз рассчитан на такую модульную интеграцию, а для Unity 6 он поддерживается.

Что я предлагаю добавить

В твоём проекте:

```
Assets/
├── CrossBound/
│   ├── Scripts/
│   │   ├── Platform/
│   │   │   └── Yandex/
│   │   │       ├── YandexGameService.cs
│   │   │       ├── YandexGameInitializer.cs
│   │   │       ├── YandexPlayerService.cs
│   │   │       ├── YandexSaveService.cs
│   │   │       ├── YandexAdsService.cs
│   │   │       ├── YandexLeaderboardService.cs
│   │   │       ├── YandexGameplayService.cs
│   │   │       └── YandexGameData.cs
```

И архитектура получится такой:

```
graph TD
    CrossBound --> IGamePlatformService
    IGamePlatformService --> YandexGameService
    YandexGameService --> Player
    YandexGameService --> Save
    YandexGameService --> Ads
    Player --> YG2_player[YG2.player]
    YG2_player --> Leaderboard
    Leaderboard --> YG2_SetLeaderboard[YG2.SetLeaderboard()]
    Save --> YG2_Storage[YG2 Storage]
    Ads --> YG2_Ads[YG2 Ads]
```

Это существенно лучше, чем, например:

```
PlayerProfileManager
{
    YG2.SetLeaderboard(...);
    YG2.RewardedAdShow(...);
    YG2.player.name;
    ...
}
```

Потому что тогда `PlayerProfileManager` перестаёт быть системой сохранения и превращается в класс, знающий о конкретной платформе.

1. YandexGameData.cs

Это наша модель данных платформы.

```
using System;

[Serializable]
public sealed class YandexGameData
{
    public bool IsInitialized;
    public bool IsAuthorized;

    public string PlayerId;
    public string PlayerName;
    public string PlayerPhoto;

    public string Platform;
```

```
}
```

2. YandexPlayerService.cs

Работа с авторизацией и данными игрока.

PluginYG2 хранит данные игрока в YG2.player: auth, name, id, photo. Если пользователь не авторизован, плагин может вернуть unauthorized или anonymous.

```
using System;
using UnityEngine;
using YG;

public sealed class YandexPlayerService
{
    public bool IsAuthorized =>
        YG2.player != null &&
        YG2.player.auth;

    public string PlayerId =>
        YG2.player != null
        ? YG2.player.id
        : string.Empty;

    public string PlayerName =>
        YG2.player != null
        ? YG2.player.name
        : string.Empty;

    public string PlayerPhoto =>
        YG2.player != null
        ? YG2.player.photo
        : string.Empty;

    public event Action PlayerDataChanged;

    public void Initialize()
    {
        YG2.onGetSDKData += OnSDKDataReceived;
    }

    public void Dispose()
    {
        YG2.onGetSDKData -= OnSDKDataReceived;
    }

    public void OpenAuthorizationDialog()
    {
        if (IsAuthorized)
            return;

        YG2.OpenAuthDialog();
    }

    private void OnSDKDataReceived()
    {
        PlayerDataChanged?.Invoke();
    }
}
```

3. YandexGameplayService.cs

Яндекс рекомендует сообщать платформе, когда игра готова, а также использовать GameplayStart и GameplayStop. В PluginYG2 это уже предусмотрено.

```
using YG;

public sealed class YandexGameplayService
{
    private bool _gameplayStarted;

    public void GameReady()
    {
        YG2.GameReadyAPI();
    }

    public void StartGameplay()
    {
        if (_gameplayStarted)
            return;

        _gameplayStarted = true;

        YG2.GameplayStart();
    }

    public void StopGameplay()
    {
        if (!_gameplayStarted)
            return;

        _gameplayStarted = false;

        YG2.GameplayStop();
    }

    public bool IsGameplayActive =>
        _gameplayStarted;
}
```

4. YandexAdsService.cs

Я бы сделал отдельную оболочку над рекламой.

PluginYG2 предоставляет `InterstitialAdShow()` для полноэкранной рекламы и `RewardedAdShow(id, callback)` для рекламы с вознаграждением.

```
using System;
using YG;

public sealed class YandexAdsService
{
    public event Action InterstitialOpened;
    public event Action InterstitialClosed;
    public event Action<string> RewardReceived;

    public void Initialize()
    {
        YG2.onOpenInterAdv += OnInterstitialOpened;
        YG2.onCloseInterAdv += OnInterstitialClosed;
        YG2.onRewardAdv += OnRewardReceived;
    }

    public void Dispose()
    {
        YG2.onOpenInterAdv -= OnInterstitialOpened;
        YG2.onCloseInterAdv -= OnInterstitialClosed;
        YG2.onRewardAdv -= OnRewardReceived;
    }

    public void ShowInterstitial()
    {
        YG2.InterstitialAdShow();
    }

    public void ShowRewarded(string rewardId, Action onReward = null)
    {
        YG2.RewardedAdShow(
            rewardId,
            () =>
            {
                onReward?.Invoke();
            }
        );
    }

    private void OnInterstitialOpened()
    {
        InterstitialOpened?.Invoke();
    }

    private void OnInterstitialClosed()
    {
        InterstitialClosed?.Invoke();
    }

    private void OnRewardReceived(string rewardId)
    {
        RewardReceived?.Invoke(rewardId);
    }
}
```

Например, в игре:

```
_yandex.Ads.ShowRewarded(
    "extra_hint",
    () =>
    {
        GiveHint();
    }
);
```

5. YandexLeaderboardService.cs

Для Cross-Bound это особенно полезно, потому что у тебя уже есть:

```
_profile.totalScore
```

PluginYG2 предоставляет:

```
YG2.SetLeaderboard(string technoName, int score);
```

При этом PluginYG2 **не сравнивает автоматически новый результат со старым**, поэтому сравнение лучше делать у нас. Также запись должна выполняться не чаще одного раза в секунду.

```
using YG;

public sealed class YandexLeaderboardService
{
    private readonly string _leaderboardName;
    private int _lastSubmittedScore;

    public YandexLeaderboardService(string leaderboardName)
    {
        _leaderboardName = leaderboardName;
    }

    public void SubmitScore(int score)
    {
        if (score <= _lastSubmittedScore)
            return;

        _lastSubmittedScore = score;

        YG2.SetLeaderboard(
            _leaderboardName,
            score);
    }
}
```

В Inspector имя таблицы я бы не хранил.

Например:

```
private const string LeaderboardName = "crossbound_score";
```

или вынес в ScriptableObject.

6. YandexSaveService.cs

Здесь важный момент.

Я бы не продолжал использовать текущий `PlayerProfileManager` как единственное хранилище.

Сейчас у тебя:

```
Application.persistentDataPath
```

и:

```
player_profile.json
```

Для WebGL это не является тем облачным сохранением, которое нам нужно от Яндекс Игр.

PluginYG2 Storage как раз предоставляет облачные сохранения; документация рекомендует использовать `Save Cloud`, а `PlayerPrefs.Save/SaveProgress` используются для записи данных.

Для твоей архитектуры я бы сделал:

```
using System;
using UnityEngine;

public sealed class YandexSaveService
{
    private const string SaveKey = "crossbound_profile";

    public void Save(PlayerProfile profile)
    {
        if (profile == null)
            return;

        string json = JsonUtility.ToJson(profile);

        PlayerPrefs.SetString(
            SaveKey,
            json);

        PlayerPrefs.Save();
    }

    public PlayerProfile Load()
    {
        if (!PlayerPrefs.HasKey(SaveKey))
            return new PlayerProfile();

        try
        {
            string json = PlayerPrefs.GetString(SaveKey);

            if (string.IsNullOrEmpty(json))
                return new PlayerProfile();

            PlayerProfile profile =
                JsonUtility.FromJson<PlayerProfile>(json);

            return profile ?? new PlayerProfile();
        }
        catch (Exception exception)
        {
            Debug.LogError(
                $"Failed to load player profile: {exception}");

            return new PlayerProfile();
        }
    }
}
```

Но здесь есть ещё один момент: если мы подключаем именно модуль **Storage PluginYG2**, я бы немного изменил этот класс, чтобы использовать механизм `Storage` непосредственно, а не дублировать его через `PlayerPrefs`.

То есть финальная версия будет:

```
PlayerProfileManager
|
v
IPlayerSaveService
|
+---- LocalPlayerSaveService
|
+---- YandexPlayerSaveService
```

Это будет правильнее.

7. YandexGameInitializer.cs

Главный объект, который создаётся на стартовой сцене.

```
using UnityEngine;

public sealed class YandexGameInitializer : MonoBehaviour
{
    public static YandexGameInitializer Instance
    {
        get;
        private set;
    }

    public YandexGameService GameService
    {
        get;
        private set;
    }

    private void Awake()
    {
        if (Instance != null)
        {
            Destroy(gameObject);
            return;
        }

        Instance = this;

        DontDestroyOnLoad(gameObject);

        GameService = new YandexGameService();

        GameService.Initialize();
    }

    private void OnDestroy()
    {
        if (Instance != this)
            return;

        GameService?.Dispose();

        Instance = null;
    }
}
```

8. YandexGameService.cs

Вот главный фасад.

```
using System;
using YG;

public sealed class YandexGameService
{
    public YandexPlayerService Player
    {
        get;
    }

    public YandexGameplayService Gameplay
    {
        get;
    }

    public YandexAdsService Ads
    {
        get;
    }

    public YandexLeaderboardService Leaderboard
    {
        get;
    }

    public bool IsInitialized =>
        YG2.isSDKEnabled;

    public string Platform =>
        YG2.platform;

    public event Action Initialized;

    public YandexGameService()
    {
        Player = new YandexPlayerService();

        Gameplay = new YandexGameplayService();

        Ads = new YandexAdsService();

        Leaderboard =
            new YandexLeaderboardService(
                "crossbound_score");
    }

    public void Initialize()
    {
        YG2.onGetSDKData += OnSDKInitialized;

        Player.Initialize();

        Ads.Initialize();
    }

    public void Dispose()
    {
        YG2.onGetSDKData -= OnSDKInitialized;
    }
}
```

```

        Player.Dispose();
        Ads.Dispose();
    }

    public void GameReady()
    {
        Gameplay.GameReady();
    }

    private void OnSDKInitialized()
    {
        Initialized?.Invoke();
    }
}

```

9. Но я бы немного переделал `ProjectStart`

Сейчас у тебя:

```

void Update()
{
    Debug.Log("Update");

    if (gameSystemsForStart.Count == 0)
    {
        isLoaded = true;
    }

    if (isLoaded)
        LoadStartScene();

    ...
}

```

Это проблемное место.

`LoadStartScene()` будет вызываться **каждый кадр**.

Кроме того, у тебя есть удаление из `List` во время `foreach`:

```

foreach (GameSystem gm in gameSystemsForStart)
{
    ...
    gameSystemsForStart.Remove(gm);
}

```

Это потенциальный `InvalidOperationException`.

Для интеграции Яндекс Игр я бы вообще заменил этот механизм.

10. Новый `ProjectStart`

```

using UnityEngine;

public sealed class ProjectStart : MonoBehaviour
{
    [SerializeField]
    private bool _waitForYandexSDK = true;

    private bool _started;

    private void Awake()
    {
        Debug.Log("CrossBound: Project Start");
    }

    private void Start()
    {
        if (!_waitForYandexSDK)
        {
            StartGame();
            return;
        }

        if (YandexGameInitializer.Instance == null)
        {
            Debug.LogError(
                "YandexGameInitializer is missing.");

            StartGame();

            return;
        }

        YandexGameInitializer.Instance
            .GameService
            .Initialized += OnYandexInitialized;
    }

    private void OnYandexInitialized()
    {
        YandexGameInitializer.Instance
            .GameService
            .Initialized -= OnYandexInitialized;

        StartGame();
    }

    private void StartGame()
    {
    }
}

```

```

    {
        if ( _started)
            Return;

        _started = true;

        var yandex =
            YandexGameInitializer.Instance?.GameService;

        yandex?.GameReady();

        SceneController.Instance
            .LoadSceneByName ("MainGameScene");
    }

    private void OnDestroy()
    {
        if (YandexGameInitializer.Instance == null)
            return;

        YandexGameInitializer.Instance
            .GameService
            .Initialized -= OnYandexInitialized;
    }
}

```

Но здесь я бы сделал ещё лучше: **не блокировать загрузку игры на SDK**. Для веб-игры лучше иметь fallback.

То есть:

```

ProjectStart
|
+---- SDK ready -----> MainGameScene
|
+---- SDK unavailable -> MainGameScene

```

А не:

```

SDK unavailable
|
X
игра
висит

```

11. Важная часть — сцена

В `ProjectLoader` я бы сделал:

```

ProjectLoader
├── ProjectStart
├── SceneController
└── YandexGameInitializer

```

`YandexGameInitializer`:

```

YandexGameInitializer
    DontDestroyOnLoad

```

Поэтому после перехода:

```

ProjectLoader
|
MainGameScene

```

он остаётся.

12. Что импортировать в PluginYG2

Тебе не нужен весь `PluginYG2`.

Согласно документации плагин модульный, и неиспользуемые модули не должны попадать в билд.

Для Cross-Bound я бы начал с:

```

PluginYourGames
├── Authorization
├── Storage
├── InterstitialAdv
├── RewardedAdv
└── Leaderboards

```

И позже:

```
Localization
EnvirData
Metrics
```

если они действительно понадобятся.

Для Authorization PluginYG2 автоматически предоставляет данные игрока через `YG2.player`.

13. Что получится в самой игре

Например, `CrosswordGameController` сейчас напрямую занимается профилем:

```
PlayerProfileManager.CompletePuzzle(
    "crossword_01",
    _score);
```

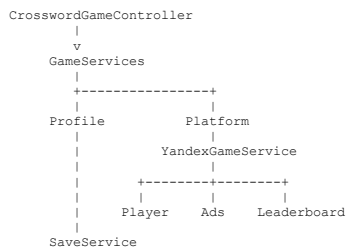
Я бы постепенно переделал это в:

```
GameServices.Profile.CompletePuzzle(
    "crossword_01",
    _score);
```

и после завершения:

```
GameServices.Platform.Leaderboard
    .SubmitScore(_score);
```

То есть:



Это даст нам нормальную платформенную архитектуру.

Что особенно важно для твоего проекта

Твой текущий `PlayerProfileManager` имеет ещё одну проблему:

```
#if UNITY_EDITOR || UNITY_STANDALONE_WIN
private static string SavePath => ...
#endif
```

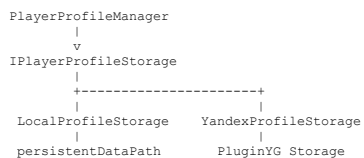
На WebGL `SavePath` вообще не определяется.

То есть текущую систему сохранений нужно переделать до WebGL-релиза.

И я бы не пытался просто добавить:

```
#if UNITY_WEBGL
...
#endif
```

в этот класс. Лучше разделить:



Так мы получим возможность запускать один и тот же Cross-Bound в Editor, WebGL и Яндекс Играх.

Ещё один момент

Я бы не добавлял пока Yandex SDK непосредственно в `CrosswordGameController`.

Вместо:

```
using YG;

...

YG2.RewardedAdvShow(...);
YG2.SetLeaderboard(...);
YG2.player.name;
```

в игровых классах будет:

```
GameServices.Platform.Ads.ShowRewarded(...);

GameServices.Platform.Leaderboard.SubmitScore(...);

GameServices.Platform.Player.PlayerName;
```

Это позволит потом без переписывания игры добавить, например:

```
Yandex Games
Crazy Games
VK Play
Steam
Desktop
```

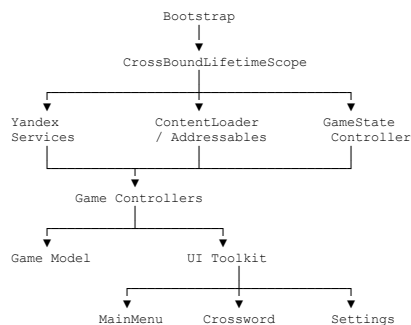
PluginYG2 сам построен вокруг подобной платформенной модели и предоставляет разные реализации платформ через свои модули.

Следующим шагом я бы сделал именно полноценный комплект `GameServices` + `IPlayerProfileStorage` + `Yandex Storage`, а затем встроил его непосредственно в твой `Cross-Bound v_0_0_1`: с конкретными путями файлов, `ProjectLoader`, `ProjectStart`, `PlayerProfileManager` и настройкой `WebGL Template`. Тогда это уже будет не просто набор классов, а готовая интеграция проекта с Яндекс Играми.

Да. И я бы использовал `VContainer`, а не `Zenject`, и сразу построил загрузку кроссвордов через `Addressables` + `async/UniTask`.

Для `Cross-Bound` это даст гораздо более чистую архитектуру, чем текущие `static Instance`, `FindObjectOfType`, ручное создание сервисов и прямые зависимости UI от `Yandex`.

1. Что я предлагаю зафиксировать



То есть:

VContainer отвечает за зависимости.

Addressables отвечает за контент и тяжёлые ресурсы.

UniTask отвечает за асинхронную загрузку.

UI Toolkit отвечает только за отображение и пользовательский ввод.

Yandex сервисы не должны торчать в игровом UI.

2. VContainer вместо Zenject

Я бы выбрал:

VContainer

Причины именно для Cross-Bound:

	Zenject	VContainer
DI	Да	Да
Unity	Отлично	Отлично
Runtime overhead	Выше	Ниже
Reflection	Больше	Меньше
Startup	Тяжелее	Легче
AOT/IL2CPP	Хорошо	Хорошо
UniTask	Хорошо	Очень хорошо
Простота	Средняя	Высокая
Под мобильные/WebGL	Хорошо	Очень хорошо
Для небольшого проекта	Может быть избыточен	Подходит отлично

Для Cross-Bound я бы **не использовал Zenject**.

И тем более не стал бы смешивать:

```
VContainer
+
Zenject
+
Singleton
```

Нужен один DI-контейнер.

У тебя уже есть опыт с VContainer, поэтому здесь логично продолжить его использование.

3. Но я бы изменил сам Bootstrap

Сейчас у нас была идея:

```
CrossBoundBootstrap
├
YandexGameInitializer
├
MainMenu
```

С VContainer лучше:

```
BootstrapScene
├
└ CrossBoundLifetimeScope
  ├── YandexGameService
  ├── PlayerProfileService
  ├── CrosswordContentService
  ├── CrossBoundGameState
  ├── AudioService
  └ SceneService
```

И дальше сцены получают зависимости через constructor injection / `IObjectResolver`.

4. Итоговая структура проекта

Я бы сейчас уже зафиксировал примерно такую структуру:

```
Assets/
├ CrossBound/
│ ├── Scripts/
│ ├── UI/
│ ├── Content/
│ ├── Addressables/
│ └ Settings/
```

Подробнее:

```
Assets/CrossBound/
├── Scripts/
│   ├── Bootstrap/
│   │   ├── CrossBoundLifetimeScope.cs
│   │   ├── CrossBoundBootstrap.cs
│   │   └── CrossBoundSceneLoader.cs
│   ├── Core/
│   │   ├── CrossBoundGameState.cs
│   │   ├── CrossBoundGameStateController.cs
│   │   └── CrossBoundResult.cs
│   ├── GameLogic/
│   │   ├── CrosswordGameController.cs
│   │   ├── CrosswordGameModel.cs
│   │   ├── CrosswordBoard.cs
│   │   ├── CrosswordCell.cs
│   │   └── CrosswordGenerator.cs
│   ├── Crossword/
│   │   ├── Data/
│   │   │   ├── CrosswordData.cs
│   │   │   ├── CrosswordEntry.cs
│   │   │   └── CrosswordDirection.cs
│   │   ├── Content/
│   │   │   ├── CrosswordAsset.cs
│   │   │   ├── CrosswordCatalog.cs
│   │   │   └── CrosswordContentService.cs
│   │   └── Validation/
│   │       └── CrosswordValidator.cs
│   ├── Player/
│   │   ├── PlayerProfile.cs
│   │   └── PlayerProfileService.cs
│   ├── Platform/
│   │   └── Yandex/
│   │       ├── YandexGameService.cs
│   │       ├── YandexPlayerService.cs
│   │       ├── YandexSaveService.cs
│   │       ├── YandexAdsService.cs
│   │       ├── YandexGameplayService.cs
│   │       ├── YandexPauseService.cs
│   │       ├── YandexLeaderboardService.cs
│   │       └── CrossBoundSavesYG.cs
│   ├── Services/
│   │   ├── ISceneService.cs
│   │   ├── SceneService.cs
│   │   ├── IAudioService.cs
│   │   └── AudioService.cs
│   └── UI/
│       ├── MainMenu/
│       │   ├── MainMenuView.cs
│       │   └── MainMenuController.cs
│       ├── Crossword/
│       │   ├── CrosswordGameView.cs
│       │   ├── CrosswordGridView.cs
│       │   ├── CrosswordCellView.cs
│       │   └── CrosswordQuestionView.cs
│       ├── Result/
│       │   └── ResultView.cs
│       └── Settings/
│           └── SettingsView.cs
├── UI/
│   ├── Documents/
│   │   ├── MainMenu.uxml
│   │   ├── CrosswordSelect.uxml
│   │   ├── CrosswordGame.uxml
│   │   ├── Result.uxml
│   │   └── Settings.uxml
│   └── Styles/
│       ├── Variables.uss
│       ├── Common.uss
│       ├── MainMenu.uss
│       ├── Crossword.uss
│       └── Settings.uss
├── Content/
│   └── Crosswords/
│       ├── crossword_001/
│       │   ├── crossword.json
│       │   └── thumbnail.png
│       ├── crossword_002/
│       │   ├── crossword.json
│       │   └── thumbnail.png
│       └── crossword_003/
│           ├── crossword.json
│           └── thumbnail.png
└── Settings/
    ├── CrossBoundConfig.asset
    └── Addressables/
```

5. Addressables — здесь я бы сделал немного иначе

Я не рекомендую делать:

```
CrosswordCatalog
├── Crossword001
├── Crossword002
├── Crossword003
├── Crossword004
└── ...
```

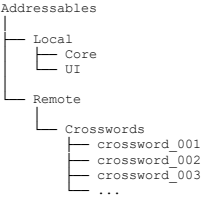
как большой ScriptableObject со всеми загруженными данными.

И не рекомендую:

Resources/
└─ Crosswords/

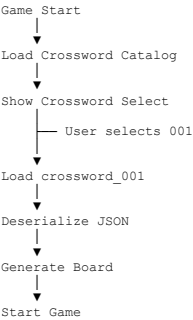
6. Нормальная схема Addressables

Я бы сделал:



При запуске игры мы не загружаем все кроссворды.

Загружается только каталог.



7. Что именно будет Addressable

Я бы сделал один CrosswordAsset.

```
using UnityEngine;
using UnityEngine.AddressableAssets;

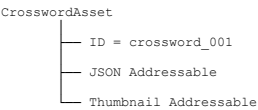
[CreateAssetMenu(
    fileName = "Crossword",
    menuName = "CrossBound/Crossword")]
public sealed class CrosswordAsset : ScriptableObject
{
    [SerializeField]
    private string _id;

    [SerializeField]
    private AssetReference _data;

    [SerializeField]
    private AssetReferenceSprite _thumbnail;

    public string Id => _id;
    public AssetReference Data => _data;
    public AssetReferenceSprite Thumbnail => _thumbnail;
}
```

Тогда:



Сам CrosswordAsset маленький.

А JSON и картинка могут находиться удалённо.

8. Но есть ещё более интересный вариант

Мы можем вообще не делать отдельный Addressable для каждого JSON через SO.

Можно использовать label:

```
crossword
crossword_001
crossword_002
crossword_003
```

Например:

```
Addressables
  Label: crossword
        crossword_001.json
        crossword_002.json
        crossword_003.json
        crossword_004.json
```

И получить каталог:

```
ReadOnlyList<TextAsset>
```

Но для игры я бы предпочёл метаданные + отдельную загрузку.

Например:

```
CrosswordCatalog
├── id
├── titleKey
├── difficulty
├── thumbnail
└── dataReference
```

И поэтому:

```
MainMenu
├── Catalog
│   ├── Crossword Card
│   │   ├── User selects
│   │   └── Load Data
```

Это позволяет сделать сотни кроссвордов без необходимости держать их все в памяти.

9. Очень важный момент: JSON

Сам JSON:

```
{
  "id": "crossword_001",
  "version": 1,
  "width": 10,
  "height": 10,
  "entries": [
    {
      "id": "q001",
      "number": 1,
      "direction": "Across",
      "startX": 0,
      "startY": 2,
      "answer": "PARIS",
      "questionKey": "crossword_001_q001"
    }
  ]
}
```

не содержит UI.

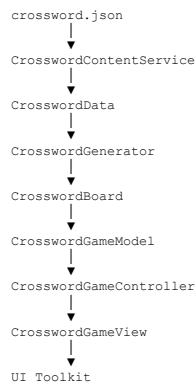
Не содержит:

```
VisualElement
Sprite
Button
Color
USS
```

Он содержит только игровые данные.

10. А UI Toolkit получает уже готовую модель

Получится:



Это очень важное разделение.

11. И здесь VContainer становится действительно полезным

Например:

```
public sealed class CrosswordGameController
{
    private readonly CrosswordContentService _content;
    private readonly CrosswordGameModel _game;
    private readonly PlayerProfileService _profile;

    public CrosswordGameController(
        CrosswordContentService content,
        CrosswordGameModel game,
        PlayerProfileService profile)
    {
        _content = content;
        _game = game;
        _profile = profile;
    }
}
```

Контроллер **не знает**, откуда появились зависимости.

Не будет:

```
CrosswordGameController.Instance
```

Не будет:

```
FindObjectOfType<CrosswordGrid>()
```

Не будет:

```
new CrosswordContentService()
```

12. Addressables тоже инжектим через сервис

Например:

```
public interface ICrosswordContentService
{
    UniTask<CrosswordData> LoadAsync(string id);
}
```

Реализация:

```
public sealed class CrosswordContentService : ICrosswordContentService
{
    public async UniTask<CrosswordData> LoadAsync(string id)
    {
        // Addressables loading.
    }
}
```

```
}  
}
```

А контроллер знает только:

```
ICrosswordContentService
```

Это уже нормальный SOLID.

13. Ещё один важный момент — Release

Addressables нельзя просто:

```
LoadAssetAsync ()
```

и забыть.

Нужно:

```
Load  
↓  
Use  
↓  
Release
```

Поэтому сервис должен контролировать lifetime:

```
UnityEngine.Task<CrosswordData> LoadAsync (...)
```

и затем:

```
Release (...)
```

Или ещё лучше — сделать handle внутри runtime-контейнера загруженного crossword.

Например:

```
CrosswordSession  
├── CrosswordData  
├── AddressableHandle  
└── Thumbnail
```

Когда пользователь выходит:

```
CrosswordSession.Dispose ()  
↓  
Addressables.Release ()
```

Это особенно важно для WebGL и мобильных устройств.

14. Что должно быть Local, а что Remote

Я бы сделал так:

Local

```
Bootstrap  
Core  
MainMenu  
UI  
Fonts  
Localization  
Common Audio  
Common VFX
```

Remote

```
Crossword JSON  
Crossword thumbnails  
Crossword-specific illustrations  
Future crossword audio  
Future events
```

То есть пользователь открывает игру:

```
~ минимальный initial download ~
  ↓
Main Menu
  ↓
Crossword Catalog
  ↓
выбрал кроссворд
  ↓
скачать только его
```

Это существенно лучше, чем положить 500 кроссвордов в initial build.

15. А можно сделать обновление контента без обновления игры

И вот здесь Addressables особенно хорошо подходит.

Например, релиз игры:

```
Game v1.0
```

имеет:

```
crossword_001
crossword_002
crossword_003
```

Через неделю добавляем:

```
crossword_004
crossword_005
crossword_006
```

и меняем remote catalog.

Игрок получает:

```
Game v1.0
+
Content v1.1
```

без нового игрового билда.

16. Я бы ещё добавил Content Version

Например:

```
{
  "id": "crossword_001",
  "contentVersion": 3,
  "width": 10,
  "height": 10,
  "entries": []
}
```

Это пригодится, если мы изменим:

```
answer
questionKey
grid
reward
difficulty
```

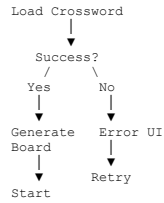
17. И главное — загрузка должна быть отказоустойчивой

Нам нельзя делать:

```
await LoadAsync();
StartGame();
```

без обработки ошибок.

Схема:



Например:

"Не удалось загрузить кроссворд."

[Повторить]

[Назад]

А если интернет пропал во время игры, текущая игра продолжает работать, потому что весь необходимый crossword уже находится в памяти.

18. Что я предлагаю сделать дальше

Теперь я бы не стал сразу писать 30 файлов вслепую.

Сначала фиксируем архитектуру:

```
VContainer
+
Addressables
+
UniTask
+
UI Toolkit
+
Yandex PluginYG2
```

И затем одним следующим блоком я могу выдать тебе **полный рабочий skeleton проекта**, начиная с:

```
01. Bootstrap
02. VContainer LifetimeScope
03. Core
04. Player
05. Yandex
06. Addressables
07. Crossword Data
08. Crossword Generator
09. Crossword Model
10. Crossword Controller
11. UI Toolkit
12. Main Menu
13. Crossword Select
14. Crossword Game
15. Result
16. Settings
17. UXML
18. USS
19. JSON
20. Scene hierarchy
21. Addressables configuration
22. VContainer configuration
```

Причём я бы сделал это **не как набор разрозненных примеров**, а как согласованный комплект, где каждый класс действительно компилируется вместе с остальными. Это позволит после этого уже перейти к реальной реализации `CrosswordGrid`, ввода букв и генерации поля.

Так пока храним 500 кроссвордов на локальной машине - это условно , и давай все листинги особенно UI и в целом как собрать приложение представь что я человек который ничего не знает про Unity

Да. Тогда я предлагаю зафиксировать **первую полноценную версию Cross-Bound** именно так:

- **Unity 6**
- **VContainer**
- **UniTask**
- **Addressables**
- **UI Toolkit**
- **PluginYG2**
- 500 кроссвордов пока **локально в билде**
- каждый кроссворд загружается **только при выборе**
- UI вообще не знает про Addressables/Yandex
- никакого Zenject
- никаких `FindObjectOfType`

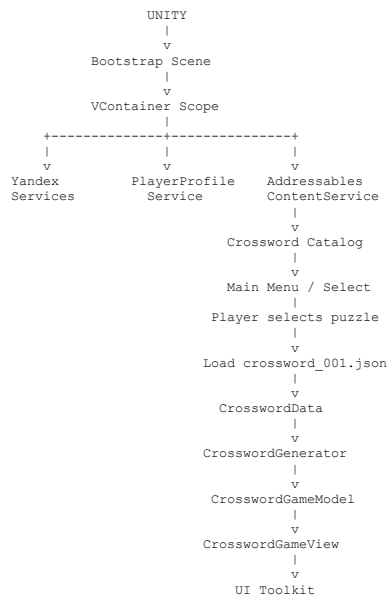
- минимум `static`
- без TMP/uGUI для нового UI

Addressables как раз позволяют загружать конкретный asset асинхронно, а после использования освобождать его handle; это нам подходит для 500 локальных JSON-файлов. VContainer имеет прямую интеграцию с UniTask и LifetimeScope. UI Toolkit в runtime строится через UIDocument + UXML + C#-поведение.

Ниже дам базовый проект целиком, так, будто Unity ты открыл впервые.

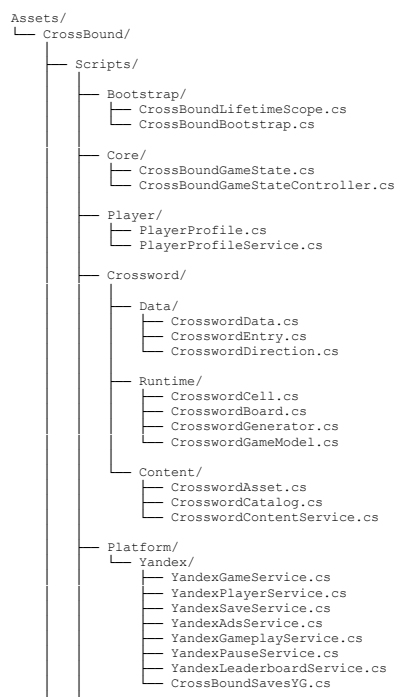
1. Что мы в итоге собираем

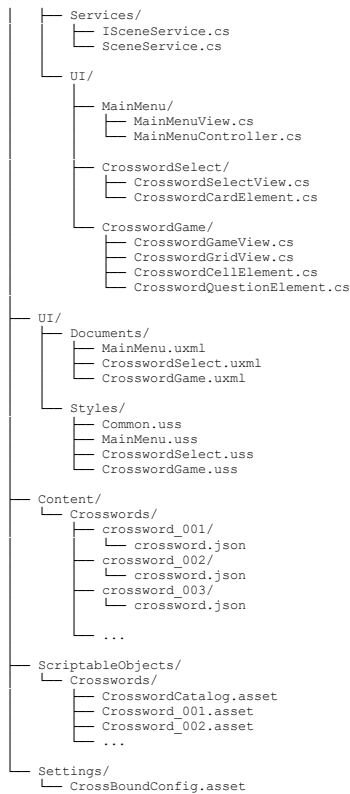
Игра будет работать так:



2. Итоговая структура проекта

Создаём:





Для 500 кроссвордов структура будет:

```
crossword_001
crossword_002
...
crossword_500
```

3. Что устанавливаем в Unity

Создай проект:

```
Unity Hub
├
└ New project
  └
    └ Unity 6
      └
        └ 3D Core
```

Название:

CrossBound

После открытия:

```
Window
├
└ Package Manager
```

Нужны:

```
Addressables
UI Toolkit
Localization
UniTask
VContainer
```

UI Toolkit уже является частью Unity, а Addressables устанавливается как пакет.

VContainer ставим через Package Manager из его официального репозитория/релиза. Официальный проект VContainer содержит `LifetimeScope`, регистрации и интеграцию с `UniTask`.

4. Сначала создаём папки

В Assets:

```
Right Click
├
Create
├
Folder
```

Создай:

CrossBound

Внутри:

```
Scripts
UI
Content
ScriptableObject
Settings
```

5. Модель кроссворда

CrosswordDirection.cs

```
public enum CrosswordDirection
{
    Across,
    Down
}
```

CrosswordEntry.cs

```
using System;

[Serializable]
public sealed class CrosswordEntry
{
    public string id;
    public int number;

    public CrosswordDirection direction;

    public int startX;
    public int startY;

    public string answer;

    public string questionKey;
}
```

CrosswordData.cs

```
using System;
using System.Collections.Generic;

[Serializable]
public sealed class CrosswordData
{
    public string id;
    public int version;

    public string title;
    public string description;

    public int width;
    public int height;

    public List<CrosswordEntry> entries =
        new List<CrosswordEntry>();
}
```

6. JSON

Создаём:

Assets/CrossBound/Content/Crosswords/crossword_001/crossword.json

Содержимое:

```
{
  "id": "crossword_001",
  "version": 1,
  "title": "Париж",
  "description": "Первый учебный кроссворд.",
  "width": 10,
  "height": 10,
```

```
"entries": [
  {
    "id": "q001",
    "number": 1,
    "direction": "Across",
    "startX": 0,
    "startY": 2,
    "answer": "PARIS",
    "questionKey": "crossword_001_q001"
  },
  {
    "id": "q002",
    "number": 2,
    "direction": "Down",
    "startX": 2,
    "startY": 0,
    "answer": "RIVER",
    "questionKey": "crossword_001_q002"
  }
]
```

Обрати внимание:

JSON не содержит клеток.

Он содержит слова.

Сетка будет построена программой.

7. Runtime клетка

CrosswordCell.cs

```
public sealed class CrosswordCell
{
    public int X { get; }
    public int Y { get; }

    public bool IsBlocked { get; set; }

    public char CorrectLetter { get; set; }

    public char CurrentLetter { get; set; }

    public CrosswordCell(int x, int y)
    {
        X = x;
        Y = y;

        IsBlocked = true;
        CorrectLetter = '\0';
        CurrentLetter = '\0';
    }
}
```

8. Board

CrosswordBoard.cs

```
public sealed class CrosswordBoard
{
    private readonly CrosswordCell[,] _cells;

    public int Width { get; }
    public int Height { get; }

    public CrosswordBoard(int width, int height)
    {
        Width = width;
        Height = height;

        _cells = new CrosswordCell[width, height];

        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                _cells[x, y] = new CrosswordCell(x, y);
            }
        }

        public CrosswordCell GetCell(int x, int y)
        {
            if (x < 0 || x >= Width)
                return null;

            if (y < 0 || y >= Height)
                return null;

            return _cells[x, y];
        }
    }
}
```

9. Генератор

CrosswordGenerator.cs

```
using UnityEngine;

public sealed class CrosswordGenerator
{
    public CrosswordBoard Generate(CrosswordData data)
    {
        if (data == null)
        {
            Debug.LogError("Crossword data is null.");
            return null;
        }

        CrosswordBoard board =
            new CrosswordBoard(
                data.width,
                data.height);

        foreach (CrosswordEntry entry in data.entries)
        {
            PlaceEntry(board, entry);
        }

        return board;
    }

    private void PlaceEntry(
        CrosswordBoard board,
        CrosswordEntry entry)
    {
        if (string.IsNullOrEmpty(entry.answer))
            return;

        for (int i = 0; i < entry.answer.Length; i++)
        {
            int x = entry.startX;
            int y = entry.startY;

            if (entry.direction == CrosswordDirection.Across)
                x += i;
            else
                y += i;

            CrosswordCell cell =
                board.GetCell(x, y);

            if (cell == null)
            {
                Debug.LogError(
                    $"Entry {entry.id} is outside board.");
                continue;
            }

            char letter =
                char.ToUpperInvariant(entry.answer[i]);

            if (!cell.IsBlocked &&
                cell.CorrectLetter != letter)
            {
                Debug.LogError(
                    $"Letter conflict at {x},{y}.");
                continue;
            }

            cell.IsBlocked = false;
            cell.CorrectLetter = letter;
        }
    }
}
```

10. Игровая модель

CrosswordGameModel.cs

```
using System;

public sealed class CrosswordGameModel
{
    private CrosswordData _data;
    private CrosswordBoard _board;

    public CrosswordData Data => _data;
    public CrosswordBoard Board => _board;

    public CrosswordEntry CurrentEntry { get; private set; }

    public event Action<CrosswordEntry> CurrentEntryChanged;
    public event Action BoardChanged;
    public event Action Completed;

    public void Initialize(
        CrosswordData data,
        CrosswordBoard board)
    {
        _data = data;
        _board = board;

        if (_data.entries.Count > 0)
            SelectEntry(_data.entries[0]);
    }

    public void SelectEntry(CrosswordEntry entry)
    {
        CurrentEntry = entry;
        CurrentEntryChanged?.Invoke(entry);
    }

    public bool SetLetter(
        int x,
        int y,
        char letter)
    {
        CrosswordCell cell =
            _board.GetCell(x, y);

        if (cell == null || cell.IsBlocked)
            return false;
    }
}
```

```

        cell.CurrentLetter =
            char.ToUpperInvariant(letter);

        BoardChanged?.Invoke();

        CheckCompleted();

        return true;
    }

    public void ClearLetter(int x, int y)
    {
        CrosswordCell cell =
            _board.GetCell(x, y);

        if (cell == null || cell.IsBlocked)
            return;

        cell.CurrentLetter = '\0';

        BoardChanged?.Invoke();
    }

    public bool IsCellCorrect(int x, int y)
    {
        CrosswordCell cell =
            _board.GetCell(x, y);

        if (cell == null || cell.IsBlocked)
            return false;

        return cell.CurrentLetter ==
            cell.CorrectLetter;
    }

    private void CheckCompleted()
    {
        for (int y = 0; y < _board.Height; y++)
        {
            for (int x = 0; x < _board.Width; x++)
            {
                CrosswordCell cell =
                    _board.GetCell(x, y);

                if (cell.IsBlocked)
                    continue;

                if (cell.CurrentLetter !=
                    cell.CorrectLetter)
                {
                    return;
                }
            }
        }

        Completed?.Invoke();
    }
}

```

11. Addressables asset

Теперь самое интересное.

Нам нужно сделать:

```

CrosswordAsset
|
+-- metadata
|
+-- JSON Addressable

```

CrosswordAsset.cs

```

using UnityEngine;
using UnityEngine.AddressableAssets;

[CreateAssetMenu(
    fileName = "Crossword",
    menuName = "CrossBound/Crossword")]
public sealed class CrosswordAsset : ScriptableObject
{
    [SerializeField]
    private string _id;

    [SerializeField]
    private string _title;

    [SerializeField]
    private string _description;

    [SerializeField]
    private int _difficulty;

    [SerializeField]
    private AssetReference _dataReference;

    public string Id => _id;
    public string Title => _title;
    public string Description => _description;
    public int Difficulty => _difficulty;

    public AssetReference DataReference =>
        _dataReference;
}

```

Почему здесь AssetReference?

Потому что Unity Addressables позволяет сохранить ссылку на конкретный asset и загружать его позднее.

12. Catalog

CrosswordCatalog.cs

```
using System.Collections.Generic;
using UnityEngine;

[CreateAssetMenu(
    fileName = "CrosswordCatalog",
    menuName = "CrossBound/Crossword Catalog")]
public sealed class CrosswordCatalog : ScriptableObject
{
    [SerializeField]
    private List<CrosswordAsset> _crosswords =
        new List<CrosswordAsset>();

    public IReadOnlyList<CrosswordAsset> Crosswords =>
        _crosswords;

    public CrosswordAsset Find(string id)
    {
        foreach (CrosswordAsset crossword in _crosswords)
        {
            if (crossword != null &&
                crossword.Id == id)
            {
                return crossword;
            }
        }

        return null;
    }
}
```

500 элементов в этом каталоге нормально.

Но мы не загружаем 500 JSON.

Мы загружаем только metadata:

```
ID
Title
Description
Difficulty
Addressable Reference
```

А JSON загружается только выбранного кроссворда.

13. Content Service

CrosswordContentService.cs

```
using System;
using System.Collections.Generic;
using Cysharp.Threading.Tasks;
using UnityEngine;
using UnityEngine.AddressableAssets;
using UnityEngine.ResourceManagement.AsyncOperations;

public sealed class CrosswordContentService
{
    private readonly CrosswordCatalog _catalog;

    private readonly Dictionary<
        string,
        AsyncOperationHandle<TextAsset>>
        _loadedAssets =
            new Dictionary<
                string,
                AsyncOperationHandle<TextAsset>>();

    public CrosswordContentService(
        CrosswordCatalog catalog)
    {
        _catalog = catalog;
    }

    public IReadOnlyList<CrosswordAsset> GetAll()
    {
        return _catalog.Crosswords;
    }

    public async UniTask<CrosswordData> LoadAsync(
        string crosswordId)
    {
        CrosswordAsset asset =
            _catalog.Find(crosswordId);

        if (asset == null)
        {
            throw new InvalidOperationException(
                $"Crossword not found: {crosswordId}");
        }

        if (!asset.DataReference.RuntimeKeyIsValid())
        {
            throw new InvalidOperationException(
                $"Invalid Addressable reference: {crosswordId}");
        }

        AsyncOperationHandle<TextAsset> handle =
            Addressables.LoadAssetAsync<TextAsset>(
                asset.DataReference);
    }
}
```



```

        TextAsset textAsset =
            await handle.ToUniTask();

        if (handle.Status !=
            AsyncOperationStatus.Succeeded)
        {
            throw new InvalidOperationException(
                $"Failed to load crossword: {crosswordId}");
        }

        _loadedAssets[crosswordId] = handle;

        CrosswordData data =
            JsonUtility.FromJson<CrosswordData>(
                textAsset.text);

        if (data == null)
        {
            Release(crosswordId);

            throw new InvalidOperationException(
                $"Invalid crossword JSON: {crosswordId}");
        }

        return data;
    }

    public void Release(string crosswordId)
    {
        if (!_loadedAssets.TryGetValue(
            crosswordId,
            out AsyncOperationHandle<TextAsset> handle))
        {
            return;
        }

        Addressables.Release(handle);
        _loadedAssets.Remove(crosswordId);
    }

    public void ReleaseAll()
    {
        foreach (var pair in _loadedAssets)
        {
            Addressables.Release(pair.Value);
        }

        _loadedAssets.Clear();
    }
}

```

Здесь есть важная вещь:

```

Load
↓
handle
↓
use
↓
Release

```

Addressables действительно требуют управления lifetime загруженных assets.

14. Player Profile

Старый `PlayerProfileManager` я бы полностью заменил.

PlayerProfile.cs

```

using System;
using System.Collections.Generic;

[Serializable]
public sealed class PlayerProfile
{
    public string playerName = string.Empty;

    public int completedPuzzles;

    public int totalScore;

    public List<string> completedPuzzleIds =
        new List<string>();

    public DateTime lastPlayed;
}

```

15. PlayerProfileService

```

using System;
using YG;

public sealed class PlayerProfileService
{
    private PlayerProfile _profile;

    public PlayerProfile Profile =>
        _profile;

    public bool IsInitialized =>
        _profile != null;

    public void Initialize()
    {
        _profile = new PlayerProfile();
    }
}

```

```

        if (!YG2.isSDKEnabled)
            return;

        LoadFromYandex();
    }

    public void CompletePuzzle(
        string crosswordId,
        int score)
    {
        EnsureInitialized();

        _profile.totalScore += score;

        if (!_profile.completedPuzzleIds.Contains(
            crosswordId))
        {
            _profile.completedPuzzleIds.Add(
                crosswordId);

            _profile.completedPuzzles++;
        }

        _profile.lastPlayed =
            DateTime.Now;

        Save();
    }

    public bool IsCompleted(string crosswordId)
    {
        EnsureInitialized();

        return _profile.completedPuzzleIds.Contains(
            crosswordId);
    }

    public void SetPlayerName(string name)
    {
        EnsureInitialized();

        _profile.playerName =
            name ?? string.Empty;

        Save();
    }

    public void Save()
    {
        EnsureInitialized();

        if (!YG2.isSDKEnabled)
            return;

        YG2.saves.crossBoundPlayerName =
            _profile.playerName;

        YG2.saves.crossBoundCompletedPuzzles =
            _profile.completedPuzzles;

        YG2.saves.crossBoundTotalScore =
            _profile.totalScore;

        YG2.saves.crossBoundCompletedPuzzleIds =
            new System.Collections.Generic.List<string>(
                _profile.completedPuzzleIds);

        YG2.saves.crossBoundLastPlayed =
            _profile.lastPlayed.ToString("O");

        YG2.SaveProgress();
    }

    private void LoadFromYandex()
    {
        _profile.playerName =
            YG2.saves.crossBoundPlayerName;

        _profile.completedPuzzles =
            YG2.saves.crossBoundCompletedPuzzles;

        _profile.totalScore =
            YG2.saves.crossBoundTotalScore;

        _profile.completedPuzzleIds =
            new System.Collections.Generic.List<string>(
                YG2.saves.crossBoundCompletedPuzzleIds);

        if (string.IsNullOrEmpty(
            YG2.saves.crossBoundLastPlayed))
        {
            return;
        }

        DateTime.TryParse(
            YG2.saves.crossBoundLastPlayed,
            out _profile.lastPlayed);
    }

    private void EnsureInitialized()
    {
        if (_profile == null)
            Initialize();
    }
}

```

16. Yandex save extension

CrossBoundSavesYG.cs

```

using System;
using System.Collections.Generic;

namespace YG
{
    public partial class SavesYG
    {
        public string crossBoundPlayerName =
            string.Empty;
    }
}

```

```

        public int crossBoundCompletedPuzzles;

        public int crossBoundTotalScore;

        public List<string> crossBoundCompletedPuzzleIds =
            new List<string>();

        public string crossBoundLastPlayed =
            string.Empty;
    }
}

```

17. Game State

CrossBoundGameState.cs

```

public enum CrossBoundGameState
{
    Boot,
    MainMenu,
    CrosswordSelect,
    Gameplay,
    Result,
    Settings
}

```

CrossBoundGameStateController.cs

```

using System;

public sealed class CrossBoundGameStateController
{
    public CrossBoundGameState State { get; private set; }

    public event Action<CrossBoundGameState>
        StateChanged;

    public CrossBoundGameStateController()
    {
        State = CrossBoundGameState.Boot;
    }

    public void SetState(
        CrossBoundGameState state)
    {
        if (State == state)
            return;

        State = state;
        StateChanged?.Invoke(state);
    }
}

```

18. Scene service

ISceneService.cs

```

using Cysharp.Threading.Tasks;

public interface ISceneService
{
    UniTask LoadAsync(string sceneName);
}

```

SceneService.cs

```

using Cysharp.Threading.Tasks;
using UnityEngine.SceneManagement;

public sealed class SceneService : ISceneService
{
    public async UniTask LoadAsync(
        string sceneName)
    {
        AsyncOperation operation =
            SceneManager.LoadSceneAsync(sceneName);

        while (!operation.isDone)
            await UniTask.Yield();
    }
}

```

19. VContainer Bootstrap

Теперь самое важное.

CrossBoundLifetimeScope.cs

```

using UnityEngine;
using VContainer;
using VContainer.Unity;

public sealed class CrossBoundLifetimeScope :

```

```

    LifetimeScope
{
    [SerializeField]
    private CrosswordCatalog _crosswordCatalog;

    protected override void Configure(
        IContainerBuilder builder)
    {
        builder.RegisterInstance(
            _crosswordCatalog);

        builder.Register<
            CrosswordContentService>(
                Lifetime.Singleton);

        builder.Register<
            CrosswordGenerator>(
                Lifetime.Singleton);

        builder.Register<
            CrosswordGameModel>(
                Lifetime.Transient);

        builder.Register<
            PlayerProfileService>(
                Lifetime.Singleton);

        builder.Register<
            CrossBoundGameStateController>(
                Lifetime.Singleton);

        builder.Register<
            SceneService>(
                Lifetime.Singleton);

        builder.Register<
            ISceneService>(
                resolver =>
                    resolver.Resolve<SceneService>(),
                Lifetime.Singleton);
    }
}

```

VContainer как раз строится вокруг `LifetimeScope` и регистрации зависимостей через `builder`.

20. Bootstrap

CrossBoundBootstrap.cs

```

using UnityEngine;
using VContainer.Unity;

public sealed class CrossBoundBootstrap :
    LifetimeScope
{
    [SerializeField]
    private CrosswordCatalog _catalog;

    protected override void Configure(
        VContainer.IContainerBuilder builder)
    {
        builder.RegisterInstance(_catalog);

        builder.Register<
            CrosswordContentService>(
                VContainer.Lifetime.Singleton);

        builder.Register<
            CrosswordGenerator>(
                VContainer.Lifetime.Singleton);

        builder.Register<
            PlayerProfileService>(
                VContainer.Lifetime.Singleton);

        builder.Register<
            CrossBoundGameStateController>(
                VContainer.Lifetime.Singleton);

        builder.Register<
            SceneService>(
                VContainer.Lifetime.Singleton);

        builder.Register<
            ISceneService>(
                resolver =>
                    resolver.Resolve<SceneService>(),
                VContainer.Lifetime.Singleton);
    }

    protected override void Awake()
    {
        base.Awake();

        DontDestroyOnLoad(gameObject);
    }
}

```

На практике я бы потом объединил эти два класса и оставил **один root LifetimeScope**. Здесь я показываю принцип, чтобы было понятно.

21. UI Toolkit

Теперь самое главное для тебя.

Мы не создаём Canvas.

Не создаём:

```
Canvas
Button
TextMeshPro
Image
```

Вместо этого:

```
UIDocument
├──
├── UXML
│   ├──
│   └── VisualElement
│       ├──
│       └── Button
│           ├──
│           └── Label
```

UIDocument **связывает** UXML с GameObject и даёт доступ к rootVisualElement.

22. Main Menu UXML

MainMenu.uxml

```
<ui:UXML
  xmlns:ui="UnityEngine.UIElements">

  <ui:VisualElement
    name="root"
    class="screen">

    <ui:VisualElement
      name="header"
      class="header">

      <ui:Label
        text="CROSS-BOUND"
        class="title" />

      <ui:Label
        name="player-name"
        text="Гость"
        class="player-name" />

    </ui:VisualElement>

    <ui:VisualElement
      name="buttons"
      class="menu-buttons">

      <ui:Button
        name="play-button"
        text="ИГРАТЬ" />

      <ui:Button
        name="settings-button"
        text="НАСТРОЙКИ" />

      <ui:Button
        name="auth-button"
        text="ВОЙТИ ЧЕРЕЗ ЯНДЕКС" />

    </ui:VisualElement>

  </ui:VisualElement>

</ui:UXML>
```

23. Main Menu USS

MainMenu.uss

```
.screen {
  flex-grow: 1;
  align-items: center;
  justify-content: center;
}

.header {
  align-items: center;
  margin-bottom: 40px;
}

.title {
  font-size: 52px;
  -unity-font-style: bold;
}

.player-name {
  margin-top: 10px;
  font-size: 22px;
}

.menu-buttons {
  width: 360px;
}

.menu-buttons Button {
```

```

height: 70px;
margin-bottom: 12px;

font-size: 24px;
-unity-font-style: bold;
}

```

24. MainMenuView

```

using UnityEngine;
using UnityEngine.UIElements;
using VContainer;

public sealed class MainMenuView :
    MonoBehaviour
{
    [SerializeField]
    private UIDocument _document;

    private Button _playButton;
    private Button _settingsButton;
    private Button _authButton;

    private MainMenuController _controller;

    [Inject]
    public void Construct(
        MainMenuController controller)
    {
        _controller = controller;
    }

    private void OnEnable()
    {
        VisualElement root =
            _document.rootVisualElement;

        _playButton =
            root.Q<Button>("play-button");

        _settingsButton =
            root.Q<Button>("settings-button");

        _authButton =
            root.Q<Button>("auth-button");

        _playButton.clicked += OnPlayClicked;
        _settingsButton.clicked += OnSettingsClicked;
        _authButton.clicked += OnAuthClicked;
    }

    private void OnDisable()
    {
        if (_playButton != null)
            _playButton.clicked -= OnPlayClicked;

        if (_settingsButton != null)
            _settingsButton.clicked -=
                OnSettingsClicked;

        if (_authButton != null)
            _authButton.clicked -= OnAuthClicked;
    }

    private void OnPlayClicked()
    {
        _controller.OpenCrosswordSelect();
    }

    private void OnSettingsClicked()
    {
        _controller.OpenSettings();
    }

    private void OnAuthClicked()
    {
        _controller.Authorize();
    }
}

```

25. MainMenuController

```

using Cysharp.Threading.Tasks;
using VContainer;

public sealed class MainMenuController
{
    private readonly ISceneService _sceneService;
    private readonly CrossBoundGameStateController _state;

    public MainMenuController(
        ISceneService sceneService,
        CrossBoundGameStateController state)
    {
        _sceneService = sceneService;
        _state = state;
    }

    public void Initialize()
    {
        _state.SetState(
            CrossBoundGameState.MainMenu);
    }

    public void OpenCrosswordSelect()
    {
        _ = LoadSelectScene();
    }

    public void OpenSettings()
    {
        _ = LoadSettingsScene();
    }

    public void Authorize()
    {
    }
}

```

```

        // Yandex authorization will be added here.
    }

    private async UniTask LoadSelectScene()
    {
        await _sceneService.LoadAsync(
            "CrosswordSelectScene");
    }

    private async UniTask LoadSettingsScene()
    {
        await _sceneService.LoadAsync(
            "SettingsScene");
    }
}

```

26. Кроссворд Select UI

Вот здесь появляется список из 500.

UXML:

```

<ui:UXML
  xmlns:ui="UnityEngine.UIElements">

  <ui:VisualElement
    name="root"
    class="screen">

    <ui:Label
      text="ВЫБЕРИТЕ КРОССВОРД"
      class="title" />

    <ui:ScrollView
      name="crossword-list"
      class="crossword-list" />

  </ui:VisualElement>
</ui:UXML>

```

27. Crossword Card

CrosswordCardElement.cs

```

using System;
using UnityEngine.UIElements;

public sealed class CrosswordCardElement :
    VisualElement
{
    public event Action Clicked;

    private readonly Button _button;

    public CrosswordCardElement(
        CrosswordAsset asset)
    {
        AddToClassList("crossword-card");

        _button = new Button();
        _button.text =
            $"{asset.Id}\n{asset.Title}\n" +
            $"Сложность: {asset.Difficulty}";

        _button.AddToClassList(
            "crossword-card-button");

        _button.clicked +=
            () => Clicked?.Invoke();

        Add(_button);
    }
}

```

28. Crossword Select View

```

using UnityEngine;
using UnityEngine.UIElements;
using VContainer;

public sealed class CrosswordSelectView :
    MonoBehaviour
{
    [SerializeField]
    private UIDocument _document;

    private CrosswordContentService _content;
    private CrosswordSelectController _controller;

    [Inject]
    public void Construct(
        CrosswordContentService content,
        CrosswordSelectController controller)
    {
        _content = content;
        _controller = controller;
    }

    private void OnEnable()
    {
        VisualElement root =
            _document.rootVisualElement;
    }
}

```

```

        ScrollView list =
            root.Q<ScrollView>{
                "crossword-list";

                foreach (CrosswordAsset asset
                    in _content.GetAll())
                {
                    CrosswordCardElement card =
                        new CrosswordCardElement(asset);

                    CrosswordAsset selectedAsset = asset;

                    card.Clicked += () =>
                    {
                        _controller.Select(
                            selectedAsset.Id);
                    };

                    list.Add(card);
                }
            }
    }
}

```

29. Controller

```

using Cysharp.Threading.Tasks;

public sealed class CrosswordSelectController
{
    private readonly ISceneService _sceneService;
    private readonly CrossBoundGameStateController _state;

    private string _selectedId;

    public CrosswordSelectController(
        ISceneService sceneService,
        CrossBoundGameStateController state)
    {
        _sceneService = sceneService;
        _state = state;
    }

    public void Initialize()
    {
        _state.SetState(
            CrossBoundGameState.CrosswordSelect);
    }

    public void Select(string crosswordId)
    {
        _selectedId = crosswordId;

        _ = StartGame();
    }

    private async UniTask StartGame()
    {
        CrosswordSession.SelectedId =
            _selectedId;

        await _sceneService.LoadAsync(
            "CrosswordGameScene");
    }
}

```

Нам понадобится маленькое хранилище выбранного crossword:

```

public static class CrosswordSession
{
    public static string SelectedId { get; set; }
}

```

Позже это тоже можно убрать в DI-сервис.

30. Crossword Game UXML

Теперь сама игра.

```

<ui:UXML
    xmlns:ui="UnityEngine.UIElements">

    <ui:VisualElement
        name="root"
        class="game-screen">

        <ui:VisualElement
            name="top-bar"
            class="top-bar">

            <ui:Label
                name="question-number"
                text="1"
                class="question-number" />

            <ui:Label
                name="score"
                text="0"
                class="score" />

        </ui:VisualElement>

        <ui:VisualElement
            name="question-panel"
            class="question-panel">

            <ui:Label

```



```

        name="question"
        text="Вопрос"
        class="question" />

</ui:VisualElement>

<ui:VisualElement
    name="grid-container"
    class="grid-container" />

<ui:VisualElement
    name="bottom-bar"
    class="bottom-bar">

    <ui:Button
        name="delete-button"
        text="УДАЛИТЬ" />

    <ui:Button
        name="hint-button"
        text="ПОДСКАЗКА" />

    <ui:Button
        name="submit-button"
        text="ПРОВЕРИТЬ" />

</ui:VisualElement>

</ui:VisualElement>

</ui:UXML>

```

31. Crossword Game USS

```

.game-screen {
    flex-grow: 1;
    padding: 20px;
}

.top-bar {
    height: 70px;
    flex-direction: row;
    justify-content: space-between;
    align-items: center;
}

.question-number {
    font-size: 28px;
}

.score {
    font-size: 28px;
}

.question-panel {
    min-height: 100px;
    margin-bottom: 20px;
    align-items: center;
    justify-content: center;
}

.question {
    font-size: 28px;
    -unity-text-align: middle-center;
}

.grid-container {
    flex-grow: 1;
    align-items: center;
    justify-content: center;
}

.bottom-bar {
    height: 80px;
    flex-direction: row;
    justify-content: center;
}

.bottom-bar Button {
    width: 180px;
    height: 55px;
    margin-left: 8px;
    margin-right: 8px;
}

```

32. Клетка UI Toolkit

```

using UnityEngine.UIElements;

public sealed class CrosswordCellElement :
    VisualElement
{
    public int X { get; }
    public int Y { get; }

    private readonly Label _letter;

    public CrosswordCellElement(
        int x,
        int y)
    {
        X = x;
        Y = y;

        AddToClassList(
            "crossword-cell");

        _letter = new Label();

        _letter.AddToClassList(
            "crossword-letter");

        Add(_letter);
    }

    public void SetBlocked()

```

```

        {
            AddToClassList("blocked");
        }

        public void SetLetter(char letter)
        {
            _letter.text =
                letter == '\0'
                ? string.Empty
                : letter.ToString();
        }

        public void SetSelected(
            bool selected)
        {
            EnableInClassList(
                "selected",
                selected);
        }
    }
}

```

33. Grid View

```

using UnityEngine.UIElements;

public sealed class CrosswordGridView :
    VisualElement
{
    private CrosswordCellElement[,] _cells;

    public CrosswordGridView(
        CrosswordBoard board)
    {
        AddToClassList(
            "crossword-grid");

        _cells =
            new CrosswordCellElement[
                board.Width,
                board.Height];

        for (int y = 0;
            y < board.Height;
            y++)
        {
            for (int x = 0;
                x < board.Width;
                x++)
            {
                CrosswordCellElement element =
                    new CrosswordCellElement(x, y);

                CrosswordCell cell =
                    board.GetCell(x, y);

                if (cell.IsBlocked)
                    element.SetBlocked();

                _cells[x, y] =
                    element;

                Add(element);
            }
        }

        public void Refresh(
            CrosswordBoard board)
        {
            for (int y = 0;
                y < board.Height;
                y++)
            {
                for (int x = 0;
                    x < board.Width;
                    x++)
                {
                    CrosswordCell cell =
                        board.GetCell(x, y);

                    _cells[x, y]
                        .SetLetter(
                            cell.CurrentLetter);
                }
            }
        }
    }
}

```

34. Grid USS

Добавляем:

```

.crossword-grid {
    flex-direction: row;
    flex-wrap: wrap;
    align-items: center;
    justify-content: center;
}

.crossword-cell {
    width: 48px;
    height: 48px;

    margin: 1px;

    border-left-width: 1px;
    border-right-width: 1px;
    border-top-width: 1px;
    border-bottom-width: 1px;

    align-items: center;
    justify-content: center;
}

```

```

.crossword-cell.blocked {
    visibility: hidden;
}

.crossword-letter {
    font-size: 28px;
    -unity-font-style: bold;
    -unity-text-align: middle-center;
}

.crossword-cell.selected {
    border-left-width: 3px;
    border-right-width: 3px;
    border-top-width: 3px;
    border-bottom-width: 3px;
}

```

Позже сделаем автоматический размер клетки:

```

available width
|
grid width / crossword width
|
cell size

```

чтобы одинаково красиво работать на:

```

PC
tablet
mobile
WebGL

```

35. CrosswordGameView

```

using UnityEngine;
using UnityEngine.UIElements;
using VContainer;

public sealed class CrosswordGameView :
    MonoBehaviour
{
    [SerializeField]
    private UIDocument _document;

    private CrosswordGameController _controller;

    private Label _question;
    private Label _questionNumber;
    private Label _score;

    private VisualElement _gridContainer;

    private Button _delete;
    private Button _hint;
    private Button _submit;

    [Inject]
    public void Construct(
        CrosswordGameController controller)
    {
        _controller = controller;
    }

    private void OnEnable()
    {
        VisualElement root =
            _document.rootVisualElement;

        _question =
            root.Q<Label>("question");

        _questionNumber =
            root.Q<Label>(
                "question-number");

        _score =
            root.Q<Label>("score");

        _gridContainer =
            root.Q<VisualElement>(
                "grid-container");

        _delete =
            root.Q<Button>(
                "delete-button");

        _hint =
            root.Q<Button>(
                "hint-button");

        _submit =
            root.Q<Button>(
                "submit-button");

        _delete.clicked +=
            _controller.DeleteLetter;

        _hint.clicked +=
            _controller.ShowHint;

        _submit.clicked +=
            _controller.Submit;

        _controller.QuestionChanged +=
            OnQuestionChanged;

        _controller.ScoreChanged +=
            OnScoreChanged;

        _controller.GridCreated +=
            OnGridCreated;
    }

    private void OnDisable()
    {

```

```

        if (_delete != null)
            _delete.Clicked -=
                _controller.DeleteLetter;

        if (_hint != null)
            _hint.Clicked -=
                _controller.ShowHint;

        if (_submit != null)
            _submit.Clicked -=
                _controller.Submit;

        if (_controller != null)
        {
            _controller.QuestionChanged -=
                OnQuestionChanged;

            _controller.ScoreChanged -=
                OnScoreChanged;

            _controller.GridCreated -=
                OnGridCreated;
        }
    }

    private void OnQuestionChanged(
        CrosswordEntry entry)
    {
        _question.Text =
            entry.QuestionKey;

        _questionNumber.Text =
            entry.Number.ToString();
    }

    private void OnScoreChanged(
        int score)
    {
        _score.Text =
            score.ToString();
    }

    private void OnGridCreated(
        CrosswordBoard board)
    {
        _gridContainer.Clear();

        CrosswordGridView grid =
            new CrosswordGridView(board);

        _gridContainer.Add(grid);
    }
}

```

36. Контроллер игры

```

using System;
using Cysharp.Threading.Tasks;

public sealed class CrosswordGameController
{
    private readonly CrosswordContentService _content;
    private readonly CrosswordGenerator _generator;
    private readonly CrosswordGameModel _model;
    private readonly PlayerProfileService _profile;
    private readonly CrossBoundGameStateController _state;

    public event Action<CrosswordEntry>
        QuestionChanged;

    public event Action<int>
        ScoreChanged;

    public event Action<CrosswordBoard>
        GridCreated;

    private int _score;

    public CrosswordGameController(
        CrosswordContentService content,
        CrosswordGenerator generator,
        CrosswordGameModel model,
        PlayerProfileService profile,
        CrossBoundGameStateController state)
    {
        _content = content;
        _generator = generator;
        _model = model;
        _profile = profile;
        _state = state;
    }

    public async UniTask InitializeAsync()
    {
        _state.SetState(
            CrossBoundGameState.Gameplay);

        CrosswordData data =
            await _content.LoadAsync(
                CrosswordSession.SelectedId);

        CrosswordBoard board =
            _generator.Generate(data);

        _model.Initialize(
            data,
            board);

        _model.CurrentEntryChanged +=
            OnQuestionChanged;

        _model.Completed +=
            OnCompleted;

        GridCreated?.Invoke(board);

        if (_model.CurrentEntry != null)
            OnQuestionChanged(
                _model.CurrentEntry);
    }

    public void DeleteLetter()

```

```

    {
        // Current selected cell will be implemented next.
    }

    public void ShowHint()
    {
        // Rewarded-ad hint will be implemented here.
    }

    public void Submit()
    {
        // Validate current answer.
    }

    private void OnQuestionChanged(
        CrosswordEntry entry)
    {
        QuestionChanged?.Invoke(entry);
    }

    private void OnCompleted()
    {
        _profile.CompletePuzzle(
            CrosswordSession.SelectedId,
            _score);

        _state.SetState(
            CrossBoundGameState.Result);
    }
}

```

37. Сейчас важный момент

Ты можешь заметить:

```
_question.text = entry.questionKey;
```

Это пока специально.

Следующим уровнем будет:

```

questionKey
{
    Unity.Localization
    {
        "Столица Франции"
    }
}

```

То есть JSON никогда не будет содержать:

```
"question": "Столица Франции"
```

Он будет содержать:

```

"questionKey":
  "crossword_001_q001"

```

Это позволит иметь:

```

Russian
English
German
French
...

```

без копирования 500 JSON.

38. Как собирается сцена

Теперь самое важное для человека, который только открыл Unity.

Scene 1

Создаём:

```
BootstrapScene
```

Hierarchy:

```

BootstrapScene
└─ CrossBound

```

На CrossBound добавляем:

CrossBoundLifetimeScope

И указываем:

Crossword Catalog

39. MainMenuScene

Создаём:

MainMenuScene

Hierarchy:

```
MainMenuScene
├── Main Camera
├── MainMenu
│   └── UIDocument
└── MainMenuController
```

UIDocument:

```
Source Asset
├──
└── MainMenu.uxml
```

И stylesheet:

MainMenu.uss

40. CrosswordSelectScene

```
CrosswordSelectScene
├── Main Camera
├── CrosswordSelect
│   └── UIDocument
```

UIDocument:

CrosswordSelect.uxml

41. CrosswordGameScene

```
CrosswordGameScene
├── Main Camera
├── CrosswordGame
│   └── UIDocument
```

UIDocument:

CrosswordGame.uxml

42. Build Settings

Открываем:

```
File
├──
└── Build Profiles
```

Добавляем:

```
0 BootstrapScene
1 MainMenuScene
2 CrosswordSelectScene
3 CrosswordGameScene
4 SettingsScene
5 ResultScene
```

Запуск:

BootstrapScene

43. Addressables

Теперь создаём:

```
Window
├
Asset Management
├
Addressables
├
Groups
```

Создаём группу:

Crosswords_Local

Все:

```
crossword_001.json
crossword_002.json
...
crossword_500.json
```

делаем Addressable.

Адреса:

```
crossword_001
crossword_002
...
crossword_500
```

44. Что реально происходит при запуске

Очень важно понять.

У нас есть:

500 JSON

Но это **не означает**:

Load 500 JSON

При запуске:

```
Bootstrap
├
Catalog
├
Main Menu
```

JSON ещё не загружены.

Игрок нажал:

Кроссворд №317

Только теперь:

```
Addressables
├
crossword_317
├
LoadAssetAsync<TextAsset>
├
JSON
├
CrosswordData
├
Generator
├
Board
├
UI
```

И когда игрок выходит:

```
Addressables.Release()
```

45. Почему это хорошая архитектура именно для 500 кроссвордов

Получаем:

```
500 crossword metadata
      |
      v
маленький Catalog
      |
      v
500 JSON
      |
      v
Addressables
      |
      v
500 thumbnails
      |
      v
Addressables
      |
      v
текущий crossword
      |
      v
RAM
```

То есть одновременно в памяти находится примерно:

```
текущий crossword
+
UI
+
общие игровые ресурсы
```

а не все 500.

46. А локальный Addressables потом легко сделать Remote

Сейчас:

```
Addressables
      |
      v
Local Build
```

Позже:

```
Addressables
      |
      v
Remote Build
      |
      v
CDN / Yandex storage / сервер
```

Код CrosswordContentService при этом практически не изменится.

Это одна из причин, почему я предлагаю **Addressables уже сейчас**, хотя пока все 500 кроссвордов локальные.

47. Что мы пока НЕ делаем

Чтобы не превратить первый запуск в монстра, пока не добавляем:

```
Save unfinished crossword
Cloud conflict resolution
Leaderboard UI
Rewarded hints
Animations
Virtualized 500-item list
Localization
Avatar download
Remote catalog
Analytics
Sound effects
Tutorial
```

Но архитектура уже готова к ним.

48. Что происходит с твоим старым проектом

Текущие классы:

```
ProjectStart.cs
PlayerProfileManager.cs
CrosswordGameController.cs
```

я бы не развивал дальше.

В частности, старый `CrosswordGameController` содержит:

```
_score = PlayerProfileManager.Profile.totalScore;
```

а потом:

```
PlayerProfileManager.CompletePuzzle(
    "crossword_01",
    _score);
```

Это приводит к тому, что накопленный общий score может снова добавляться как score текущей партии.

Мы это исправили архитектурно:

```
PlayerProfile.totalScore
+
currentPuzzleScore
```

отдельно.

49. Итоговая ответственность классов

Вот это я предлагаю считать главным правилом проекта:

Класс	Что делает
<code>CrosswordData</code>	данные JSON
<code>CrosswordAsset</code>	metadata + Addressable reference
<code>CrosswordCatalog</code>	список 500 кроссвордов
<code>CrosswordContentService</code>	загрузка Addressables
<code>CrosswordGenerator</code>	строит поле
<code>CrosswordBoard</code>	runtime-сетка
<code>CrosswordCell</code>	одна клетка
<code>CrosswordGameModel</code>	состояние текущей игры
<code>CrosswordGameController</code>	игровая логика
<code>CrosswordGameView</code>	UI
<code>CrosswordGridView</code>	визуальная сетка
<code>CrosswordCellElement</code>	визуальная клетка
<code>PlayerProfileService</code>	прогресс игрока
<code>YandexGameService</code>	фасад Yandex
<code>SceneService</code>	переходы между сценами
<code>VContainer</code>	создание зависимостей
<code>Addressables</code>	загрузка контента
<code>UI Toolkit</code>	интерфейс

И самое главное:

```
UI
├── X
│   ├── Addressables
│   ├── Yandex
│   ├── JSON
│   └── PlayerProfile
```

UI общается только с Controller/ViewModel-подобным слоем.

Следующий этап

Я бы теперь не добавлял ещё функциональности поверх этого skeleton. Следующий шаг стоит сделать практически: собрать этот проект в Unity с нуля по кликам, включая установку VContainer, создание CrosswordCatalog.asset, превращение crossword_001.json в Addressable, создание UIDocument, подключение UXML/USS и запуск BootstrapScene.

После этого уже следующим блоком можно сделать полноценный ввод букв мышью/клавиатурой, выбор клетки, выбор вопроса, проверку слова, подсветку ошибок, подсказку и завершение кроссворда.

вторник 15:09

вот алгоритм

1. <https://habr.com/ru/articles/249899/>

2. второе целевые языки пока только английский и русский

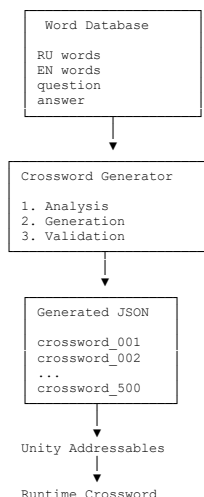
Да. Тогда я бы изменил предыдущую архитектуру генератора: не делать простой PlaceEntry() по заранее заданным координатам, а строить генератор именно на основе алгоритма из статьи.

Статья описывает двухэтапную схему: сначала анализ и построение порядка генерации, затем перебор с откатами; дополнительно используются фрагменты, шаблоны и акселераторы.

И второй важный момент: целевые языки только русский и английский. Это существенно влияет на словарь, нормализацию букв и эвристики.

Как предлагаю построить Cross-Bound

Я бы разделил систему на две совершенно разные части:



То есть Unity не должна каждый раз генерировать кроссворд во время игры.

Генерация будет выполняться заранее, например editor/tool:

```
Dictionary
↓
Generator
↓
500 готовых crossword.json
↓
Addressables
↓
Unity Game
```

Это особенно важно для мобильных/WebGL/Yandex Games.

1. Что именно берём из статьи

Не нужно буквально копировать алгоритм статьи. Мы используем его как основу.

Основные идеи:

Этап А — Analysis

Для каждого слова определяем:

```
length
intersection count
installation difficulty
possible intersections
fragment membership
generation priority
```

Например:

```
ELEPHANT
length = 8
intersections = 6
difficulty = 8 * 6 = 48
```

А:

```
CAT
length = 3
intersections = 1
difficulty = 3
```

Получаем порядок примерно:

```
ELEPHANT
COMPUTER
LANGUAGE
...
HOUSE
CAT
DOG
```

То есть сначала ставим сложные слова, а короткие оставляем на конец.

Это соответствует решениям №1 и №2 из статьи.

2. Generation

После анализа начинаем реальный backtracking:

```
Place word #1
|
Place word #2
|
Place word #3
|
No candidates
|
BACKTRACK
|
Change previous intersecting word
|
Try again
```

Например:

```
      C
      A
COMPUTER ----T
      S
```

Допустим, следующее слово должно пересекаться:

```
      ?
      A
      ?
      T
```

Из словаря выбираем кандидатов.

Если:

```
candidate #1 -> conflict
candidate #2 -> conflict
candidate #3 -> conflict
...
candidate #17 -> OK
```

ставим candidate #17.

Если вариантов вообще нет:

```
BACKTRACK
```

и меняем слово, которое создало ограничение.

Это центральная часть алгоритма статьи.

3. Шаблоны

Вот эту часть я бы **обязательно реализовал**.

Например, после размещения:

С А Т

пересекающееся слово может получить шаблон:

A

И вместо перебора всех английских слов:

APPLE
BEAR
COLD
DARK
GAME
HARD
...

мы сразу ищем:

? A ? ?

Например:

BANK
CARD
DARK
FARM
GAME
HARD

Внутри генератора это будет примерно:

Pattern:
A

и:

```
WordIndex.Find(pattern)
```

4. Но я бы сделал шаблоны лучше, чем в статье

Для нас есть два языка:

RU
EN

Поэтому шаблон должен работать с **алфавитом конкретного языка**.

English

ABCDEFGHIJKLMNOPQRSTUVWXYZ

Russian

АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ

Например:

RU:
П_РИ_
EN:
С_МР_Т_Р

При этом генератор вообще не должен знать, что такое русский или английский.

Он получает:

LanguageDefinition

и работает с ним.

5. Я бы ввёл LanguageDefinition

Например:

```
public sealed class CrosswordLanguageDefinition
{
    public string Id { get; }
    public string Alphabet { get; }

    public CrosswordLanguageDefinition(
        string id,
        string alphabet)
    {
        Id = id;
        Alphabet = alphabet;
    }
}
```

И две конфигурации:

```
English
ID = en
Alphabet = ABCDEFGHIJKLMNOPQRSTUVWXYZ

Russian
ID = ru
Alphabet = АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ
```

Это позволит нам позже добавить:

```
de
fr
es
it
```

без переписывания генератора.

6. Очень важный момент с русским языком

Для русского я бы **не объединял автоматически**:

Е = Ё

и:

Ь = ъ

Потому что это уже меняет корректность слов.

На уровне словаря можно иметь отдельную нормализацию:

ЁЛКА

и:

ЕЛКА

но решение должно быть явно задано политикой языка.

Например:

```
public sealed class CrosswordLanguageRules
{
    public bool IgnoreYo { get; }
    public bool AllowSoftSignAtStart { get; }
    public bool AllowHardSignAtStart { get; }
```

```
}    ...  
}
```

Для первой версии я бы сделал:

```
RU:  
Е != Е  
Ь != Ь  
  
EN:  
A-Z
```

а ограничения типа:

```
Ь не может быть первым  
Ъ не может быть первым
```

вынес бы в `LanguageRules`.

7. База слов

Вот здесь начинается важнейшая часть проекта.

Статья отдельно подчёркивает, что создание и очистка базы слов занимало огромную часть работы.

Нам не нужно делать одну базу:

```
words.txt
```

Лучше:

```
Content/  
└─ Dictionary/  
    └─ en/  
        ├── words.json  
        └── questions.json  
    └─ ru/  
        ├── words.json  
        └── questions.json
```

Но для генератора я бы вообще использовал **индексированную базу**.

8. WordEntry

Например:

```
[Serializable]  
public sealed class WordEntry  
{  
    public string id;  
    public string answer;  
    public string questionKey;  
    public string language;  
}
```

Например:

```
{  
    "id": "en_computer",  
    "answer": "COMPUTER",  
    "questionKey": "word.computer",  
    "language": "en"  
}
```

Русский:

```
{  
    "id": "ru_компьютер",  
    "answer": "КОМПЬЮТЕР",  
    "questionKey": "word.computer",  
    "language": "ru"  
}
```

При этом **ответ и вопрос — разные сущности**.

Это позволит нам сделать:

English:
What is a machine used for computing?
→ COMPUTER

Russian:
Электронное устройство для вычислений
→ КОМПЬЮТЕР

9. Индекс слов

Для скорости я бы не делал:

```
foreach (Word word in allWords)
```

при каждом поиске.

Для 100 000 слов это станет очень неприятно.

Нужен индекс:

```
length
+
letter position
+
letter
```

Например:

```
5 letters
position 0 = A
```

получаем:

```
ABOUT
APPLE
ADULT
AGENT
...
```

А для:

```
_ A _ E _
```

можно пересечь несколько индексов.

Архитектурно:

```
WordDatabase
├── ByLength
├── ByPosition
└── PatternIndex
```

10. Я бы сделал отдельный Editor Generator

Это очень важно.

Не надо помещать тяжёлый алгоритм в runtime:

```
Assets/CrossBound/Scripts/
```

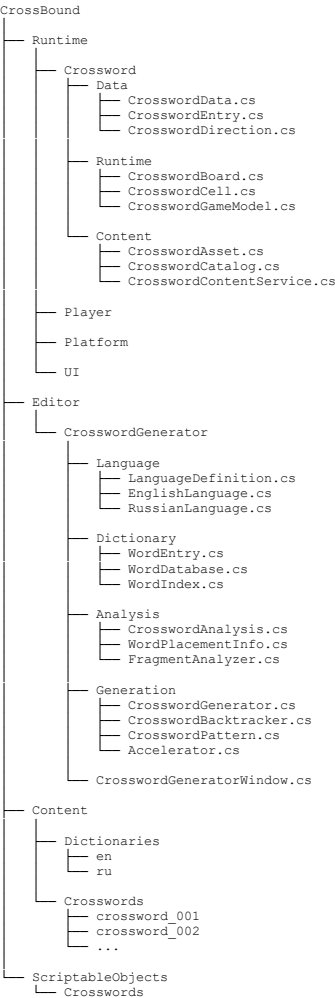
Лучше:

```
Assets/
├── CrossBound/
│   ├── Editor/
│   │   ├── CrosswordGenerator/
│   │   │   ├── CrosswordGeneratorWindow.cs
│   │   │   ├── CrosswordAnalysis.cs
│   │   │   ├── CrosswordBacktracker.cs
│   │   │   ├── CrosswordPattern.cs
│   │   │   ├── CrosswordWordDatabase.cs
│   │   │   └── CrosswordGeneratorSettings.cs
│   └── Runtime/
│       └── Crossword/
```

Runtime тогда вообще не знает о тяжёлом генераторе.

11. Полная схема

Я бы сейчас окончательно разделил проект так:



12. И ещё одно изменение относительно предыдущего варианта

В предыдущей версии у нас было:

```
CrosswordGenerator.Generate(CrosswordData data)
```

и CrosswordData уже содержал:

```
startX
startY
direction
```

Это подходит для готового кроссворда, но не для генератора.

Теперь я бы разделил:

Input

```
WordDatabase
+
GridSettings
+
Language
```

Generator


```
WordDatabase
├
Analysis
├
PlacementPlan
├
Backtracking
├
GeneratedCrossword
```

Output

CrosswordData

И уже CrosswordData содержит:

```
{
  "id": "crossword_001",
  "language": "en",
  "width": 15,
  "height": 15,
  "entries": [
    {
      "id": "word_001",
      "number": 1,
      "direction": "Across",
      "startX": 2,
      "startY": 5,
      "answer": "COMPUTER",
      "questionKey": "word.computer"
    }
  ]
}
```

То есть координаты появляются только после работы генератора.

13. Итого

Я предлагаю сейчас зафиксировать следующие требования:

Компонент	Решение
Алгоритм	Основа — алгоритм из Habr
Генерация	Editor-time
Перебор	Backtracking
Анализ	Да
Сортировка слов	Длина + сложность
Фрагменты	Да
Шаблоны	Да
Акселераторы	Да, после базовой версии
Языки	RU + EN
Runtime generation	Нет
Готовые кроссворды	JSON
Количество	500
Хранение	Addressables
Загрузка	Только выбранный crossword
UI	UI Toolkit
DI	VContainer
Yandex	PluginYG2
Localization	RU/EN
Unity	Unity 6

И это хорошо сочетается с твоей идеей **500 локальных кроссвордов**: генератор один раз тяжело работает в Editor, а игра получает уже готовые сетки и практически не тратит CPU на их построение.

Следующим шагом я бы сделал **не UI и не Yandex**, а сам генератор: WordEntry → WordDatabase → CrosswordPattern → Analysis → Backtracking, **сначала** на маленьком словаре из 20–30 русских/английских слов. После того как он стабильно строит сетку, подключим его к генерации 500 JSON и Addressables.